Agentic AI

Assignment 3

Author: Prashant Meena (24b0949)

July 1, 2026

Figure 1: Chatbot Image

Solution 1

Link of my Google Drive (All 3 Questions)

Code File

I have implemented a ReAct agent which has access to 3 tools. It uses Wikipedia for info related to facts or persons or places and if we need to search something that might not be on wikipedia then it uses DUCKDUCKGO Search. It can also use Calculator btw.

Figure 2: Terminal Output

```
from langchain_community.tools import DuckDuckGoSearchRun
from langchain.agents import initialize_agent, AgentType
from langchain_groq import ChatGroq
from langchain.tools import tool
from dotenv import load_dotenv
import requests
import os

load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    api_key=GROQ_API_KEY,
)

search = DuckDuckGoSearchRun()

@tool
def calc_tool(expr: str) -> str:
```

```
    """Use only and only for arithmetic expressions like 2+2 or 25*(3+2)."""
    return str(eval(expr))

@tool
def search_tool(query: str) -> str:
    """Use for current events, weather, recent facts, news."""
    return search.run(query)

@tool
def wiki_tool(query: str) -> str:
    """
    Use for exact entity names only.
    Examples:
    Napoleon
    Adolf Hitler
    World War II
    """
    try:
        headers = {
            "User-Agent": "ReAct/1.0"
        }

        search_url = "https://en.wikipedia.org/w/api.php"

        params = {
            "action": "query",
            "prop": "extracts",
            "exintro": True,
            "explaintext": True,
            "titles": query,
            "format": "json"
        }

        r = requests.get(search_url, params=params, headers=headers, timeout=10)
        r.raise_for_status()

        data = r.json()
        pages = data["query"]["pages"]

        page = next(iter(pages.values()))

        if "missing" in page:
            return f"No Wikipedia page found for: {query}"

        text = page.get("extract", "").strip()

        if not text:
            return "No summary available."

```

```python
        return text[:1000]

    except Exception as e:
        return "Wikipedia API failed."

tools = [search_tool, wiki_tool, calc_tool]

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True,
    handle_parsing_errors=True
)

print("Agent: Hello! Ask me something (Enter 'q' to quit).")
while True:
    query = input("\nUser: ").strip()

    if query == 'q':
        print("Agent: Goodbye!")
        break

    result = agent.run(query)
    print("Agent:", result)
```

Solution 2

I have implemented a LangGraph based Research agent which uses a local ollama based model (slower and weaker than groq based models).

```python

from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_ollama import ChatOllama

llm = ChatOllama(model="gemma2:2b")

class AgentState(TypedDict):
        question: str
        answer: str
        score: int
        iterations: int

def generate(state):
        print("Entered generate")
        prompt = f"Answer: {state['question']}"
        response = llm.invoke(prompt)
        return {
                "answer": response.content,
```

```
                "iterations": state["iterations"] + 1
        }

def evaluate(state):
        print("Entered evaluate")
        answer = state["answer"]
        words = len(answer.split())

        score = state["score"]
        if words > 100:
                score += 1

        return {"score": score}

def refine(state):
        print("Entered refine")
        response = llm.invoke(f"Improve this:\n{state['answer']}")
        return {"answer": response.content}

def route(state):
        if state["score"] >= 1:
                print("Routed to end")
                return "end"
        print("Routed to refine")
        return "refine"

builder = StateGraph(AgentState)

builder.add_node("generate", generate)
builder.add_node("evaluate", evaluate)
builder.add_node("refine", refine)

builder.set_entry_point("generate")

builder.add_edge("generate", "evaluate")
builder.add_edge("refine", "evaluate")

builder.add_conditional_edges(
    "evaluate",
    route,
    {
        "end": END,
        "refine": "refine"
    }
)

graph = builder.compile()

query = input("Enter your topic: ")
initial_state = {
```

```python
    "question": query,
    "answer": "",
    "score": 0,
    "iterations": 0
}

result = graph.invoke(initial_state)
print(result)
```

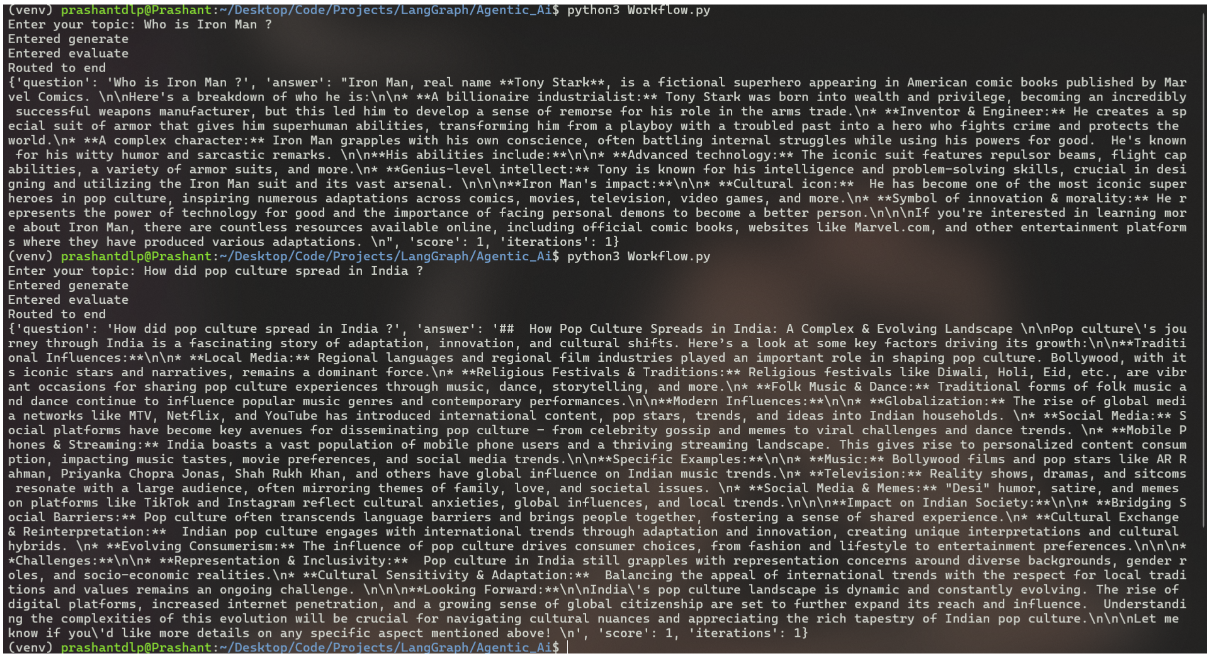

Figure 3: Output of my silly little gemma2:2b

Solution 3

```python
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
        {
            "gmail": {
                "command": "npx",
                "args": ["@gongrzhe/server-gmail-autoauth-mcp"],
                "transport": "stdio",
            }
        }
)

from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_groq import ChatGroq # fast
# from langchain_ollama import ChatOllama # slow af
from dotenv import load_dotenv
```

```python
import asyncio
import os

load_dotenv()

# GROQ_API_KEY = os.getenv("GROQ_API_KEY")

llm = ChatGroq(model="llama-3.3-70b-versatile") #, api_key = GROQ_API_KEY)

class AgentState(TypedDict):
        context: str # recipient name & email, sender name & email, context string
        draft: str
        score: int # heuristic for exiting the loop
        iterations: int

def generate(state: AgentState):

        prompt = f"""
                Write a professional cold email based on the context below.

                Context:{state["context"]}

                Sender Name: Prashant Meena

                Requirements:
                - Write only the final email body
                - No placeholders (like [Name], YOUR_NAME, COMPANY)
                - No notes, explanations, or markdown
                - Concise, personalized, and professional
                - Sound natural and human, not robotic
                - Clear reason for outreach
                - End with a simple call-to-action
                - Sign off with: Prashant Meena

                This email will be sent exactly as generated, so make it fully ready to send.
        """

        response = llm.invoke(prompt)

        return {
                "draft": response.content
        }

def review(state: AgentState):
        prompt = f"""
        Review this email and score it from 1-10.
```

```
        Email:
        {state["draft"]}

        Score based on:
        - clarity
        - professionalism
        - grammar
        - call to action

        Return ONLY integer score.
        """

        response = llm.invoke(prompt)

        try:
                score = int(response.content.strip())
        except:
                score = 5

        return {
                "score": score,
                "iterations": state["iterations"] + 1
        }

def refine(state: AgentState):
        prompt = f"""
        Improve this email to get score 9+.

        Current email:
        {state["draft"]}
        """

        response = llm.invoke(prompt)

        return {
                "draft": response.content
        }

def route(state: AgentState):
        if state["score"] >= 8:
                return "end"

        if state["iterations"] >= 3:
                return "end"

        return "refine"

# initial -> generate -> review -> end($)
#                ^           |
#                |           |
```

```python
#                <------------

builder = StateGraph(AgentState)

builder.add_node("generate", generate)
builder.add_node("review", review)
builder.add_node("refine", refine)

builder.set_entry_point("generate")

builder.add_edge("generate", "review")
builder.add_edge("refine", "review")

builder.add_conditional_edges(
        "review",
        route,
        {
                "end": END,
                "refine": "refine"
        }
)

graph = builder.compile()

# sender = input(), we already know
recipient_name = input("Enter Recipient's Name: ")
recipient_email = input("Enter Recipient's Valid Gmail: ")
email_context = input("Provide some context regarding Email: ")


initial_state = {
        "context": f"""
                recipient_name: {recipient_name},
                recipient_email: {recipient_email},
                email_context:{email_context}
        """,
        "draft": "",
        "score": 0,
        "iterations": 0
}
result = graph.invoke(initial_state)

async def send(final_draft: str): # USES MCP SERVER
        global recipient_name, recipient_email

        tools = await client.get_tools()

        print("Available tools:")
        for tool in tools:
                print(tool.name)

```

```python
        for tool in tools:
            if "send" in tool.name.lower():
                response = await tool.ainvoke({
                    "to": [recipient_email],
                    "subject": "Quick Introduction",
                    "body": final_draft
                })
                print(response)
                return

        print("No send tool found.")

asyncio.run(send(result["draft"]))
```

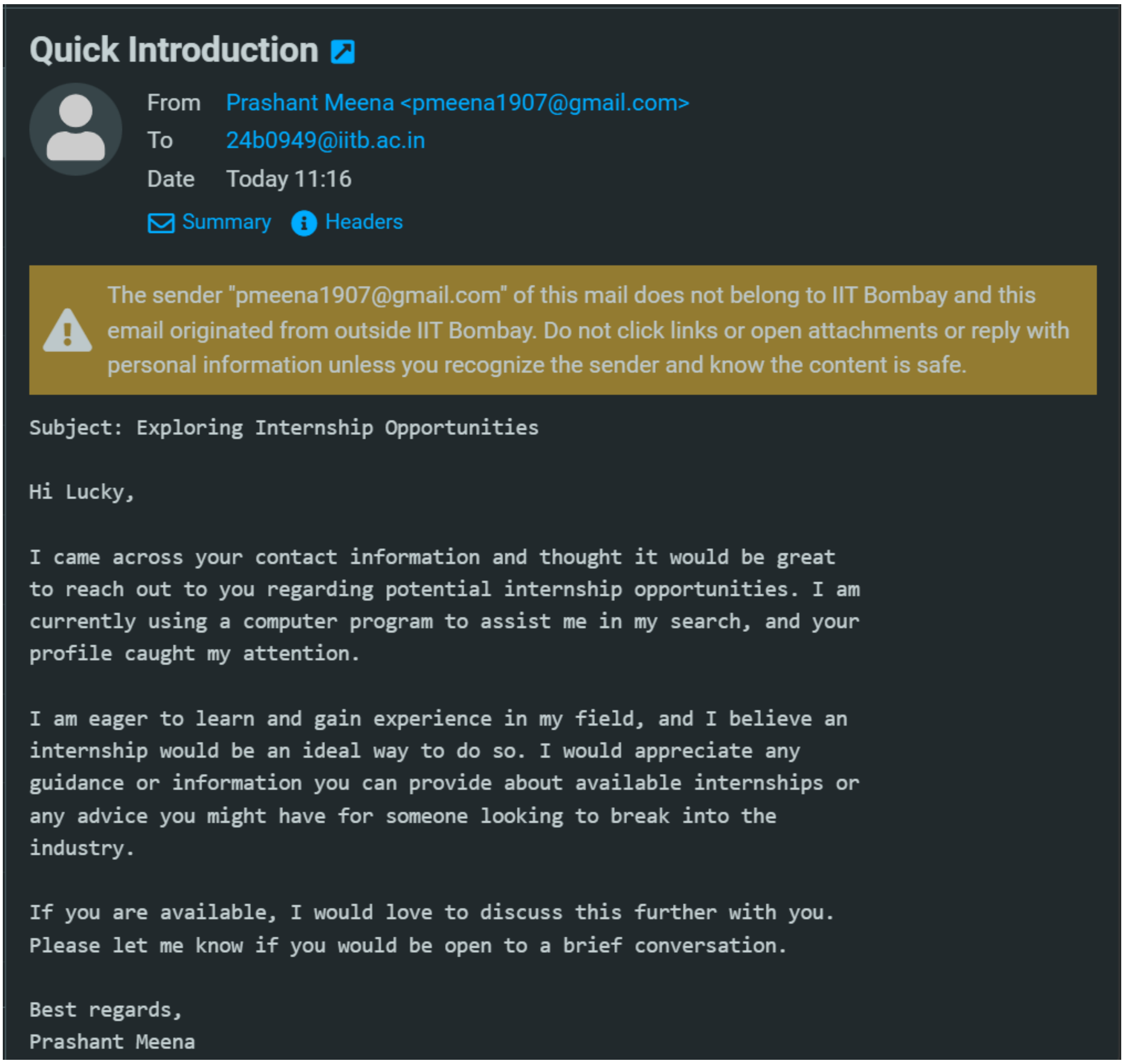

Figure 4: EMail Proof